

# Lab6 实验报告

---

## 一、思考题

### Thinking 6.1

如果不关闭自己不会用到的管道端，会产生两类问题：

- 读端无法收到 EOF 信号，可能一直阻塞等待数据；
- 写端因为对端仍然打开，无法判断peer是否结束，导致资源无法及时释放。

因此必须在父/子进程中关闭无用端，否则会出现管道死锁和引用计数判断错误。

### Thinking 6.2

要让父进程作为管道“读者”，只需交换父子进程中读写端的角色：

```
if (fork() == 0) {
    /* 子进程写 */
    close(fildes[0]);
    write(fildes[1], "Helloworld\n", 12);
    close(fildes[1]);
    exit(EXIT_SUCCESS);
} else {
    /* 父进程读 */
    close(fildes[1]);
    read(fildes[0], buf, 100);
    printf("parent-process read:%s", buf);
    close(fildes[0]);
    exit(EXIT_SUCCESS);
}
```

这样父进程关闭写端并读取，子进程关闭读端并写入，保证管道端被正确使用。

### Thinking 6.3

在 `lab-solution/mips32/user/lib/pipe.c` 中，`PIPE_SIZE` 只有 32，这会导致：

- 当写入数据较多时，管道很快被填满；
- 当读取速度较慢时，写端频繁 `syscall_yield`；
- 当读取速度较快时，读端频繁等待数据。

因此缓冲区太小时，吞吐率下降，进程切换增加；适度增大缓冲区可以减少切换，提高连续读写效率，但会消耗更多内存。

### Thinking 6.4

`pageref(rfd) + pageref(wfd) = pageref(pipe)` 不是恒等式。

- 这条关系只在管道的读写端和 pipe 数据页的映射引用计数完全一致时成立；
- 在 `dup / close` 等多步操作过程中，引用计数会经历中间状态；
- 如果在这些中间状态读取计数，等式可能失效，导致 `_pipe_is_closed` 判断出现误判。

因此该公式只能作为“期望关系”，不能在并发修改时当作严格判断依据。

## Thinking 6.5

`user/lib/fd.c` 中 `dup()` 的实现就是一个典型的非原子多步操作：

```
close(newfdnum);
/* map oldfd data pages to newfd data pages */
/* map oldfd fd 页到 newfd fd 页 */
```

若另一个进程同时对同一 pipe 执行 `close()` 或 `dup()`，则 pipe 数据页和 fd 页的引用计数可能暂时不同步，产生竞态，导致预期之外的行为。

## Thinking 6.6

MOS 中的系统调用进入内核时对当前 env 的 trap 处理通常是原子的，但“用户态的系统调用序列”并非原子。

- 例如 `pipe()` 包含多次 `syscall_mem_alloc`、`syscall_mem_map`；
- `dup()` 先 `close(newfdnum)`，再映射数据页，再映射 fd 页；
- 这类函数在用户态层面不是一个不可分割的原子操作。

所以系统调用的单次内核陷入可能是原子，但用户级封装的复合操作不一定是原子。

## Thinking 6.7

按照 `pipe_close` 中先关闭 fd 再解除 pipe 映射的顺序，确实可以缩短竞争窗口，但不能彻底消除竞态。

- 先解除 fd 映射可以让 `pageref(fd)` 计数更快变化；
- 但 `pipe` 数据页的引用计数仍然在之后解除时变化，两个计数之间仍存在短暂不一致。

`fd.c` 中的 `dup()` 如果复制的 fd 指向管道，同样会出现类似问题：它先 `close(newfdnum)`，再映射 pipe 数据页，再映射 fd 页，若与另一个进程的 `close` 交错，就可能导致引用计数中间状态和竞态判断错误。

## Thinking 6.8

- Lab5 打开文件时，用户态先调用 `open()`，通过 IPC 请求 fs server 读取目录项并分配 `Fd`；
- fs server 返回 `Filefd` 页后，用户进程将该页映射成 `Fd`，形成文件描述符与文件数据的映射关系。
- Lab1/Lab3 读取 ELF 时，`elf_load_seg()` 处理 `PT_LOAD` 段，并按页面调用回调函数加载数据；
- `load_icode_mapper()` 为每个段分配页面，并在 `src != NULL` 时拷贝内容；
- 对于 bss 段，`p_filesz < p_memsz` 时，`elf_load_seg()` 会给没有文件数据的区域传入空内容，`load_icode_mapper()` 只分配页面并保持其初值为 0。

因此 ELF 中 `bss` 段虽然在文件中不占空间，但在加载到内存后会分配页面并初始化为 0，从而占据虚拟地址空间。

## Thinking 6.9

标准输入输出 0 和 1 的安排发生在 `init` 启动 shell 之前，而不是重定向时首次创建。

在 `lab-solution/mips32/user/init.c` 中：

```
if ((r = opencons()) != 0) user_panic("opencons: %d", r);
if ((r = dup(0, 1)) < 0) user_panic("dup: %d", r);
```

这保证了 shell 启动时已经有 fd 0 和 fd 1。之后 shell 的重定向通过 `dup(fd, 0) / dup(fd, 1)` 将文件或管道端口映射到标准输入/输出。

## Thinking 6.10

- `ls.b` 和 `cat.b` 是外部命令，MOS shell 通过 `spawn()` 创建子进程执行它们；
- 如果是内部命令，shell 本身会直接执行，不需要新进程。

Linux 中 `cd` 是内部命令，因为它要修改当前 shell 进程的工作目录；如果用外部命令实现，工作目录变化只会影响子进程，而不会影响父 shell。

## Thinking 6.11

命令 `ls.b|cat.b>motd` 中：

- `ls.b` 作为管道左侧程序会被 `spawn()`；
- `cat.b` 作为管道右侧程序也会被 `spawn()`。

因此可以观察到 2 次 `spawn()`，对应 `ls.b` 和 `cat.b`；也会有 2 次子进程销毁，分别是这两个命令执行结束后的回收。

---

## 二、难点分析与实验体会

### 1. 核心难点

- 理解管道读写端的引用计数和 `pageref` 判断逻辑。
- 分析 `dup`、`close` 的多步映射顺序，识别其中的竞态条件。
- 对比 `pipe` 和 `spawn` 的用户态实现，理解系统调用与用户封装之间的边界。

### 2. 实验收获

- 管道必须关闭不使用的端，否则会造成死锁和资源无法回收。
- `dup` 不是简单复制，而是通过内存映射共享 fd 和数据页。
- ELF 加载中 `bss` 段通过分配零页保证内存空间占用且初始化为 0。

### 3. 实验注意事项

- 管道读写前先关闭无用端，避免遗留文件描述符。
  - 重定向时使用 `dup(fd, 0)/dup(fd, 1)`，不要直接修改标准 fd。
  - 读取 ELF 和 spawn 时，必须检查 `readn`、`seek` 和 `syscall_mem_map` 的返回值。
- 

### 三、原创说明

本报告基于本人实验过程、课程 MOS Lab5 代码阅读与调试整理完成，个人笔记+AI整理，部分概念表述参考实验指导书和源码注释。